

Dr. Isah Charles Saidu
Lead Instructor
Liatris Systems

AI & Technology Solutions

Session Duration: 2 Hours

July 5, 2026



Liatris Systems



In partnership with the Independent National Electoral Commission

Session Roadmap

Part 1: Generative AI

- ⚙️ What is Generative AI?
- 🏗️ How It Works
- ★ Key Models & Capabilities

Part 2: Large Language Models

- ⚙️ What are LLMs?
- 📋 Major LLMs
- ☁️ Local LLMs with Ollama

Part 3: Prompt Engineering

- ⚙️ What is Prompt Engineering?
- ✅ Best Practices
- 🔗 Advanced Techniques

Part 4: INEC Applications

- 💬 Voter Education Chatbot
- 🛡️ Misinformation Detection
- 🔑 Implementation

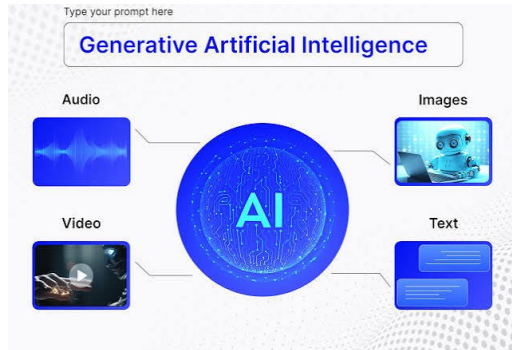
What is Generative AI?

What is Generative AI?

Generative AI is a type of artificial intelligence that can **create new content** – text, images, audio, code, and video – based on patterns learned from training data.

What Makes It "Generative"?

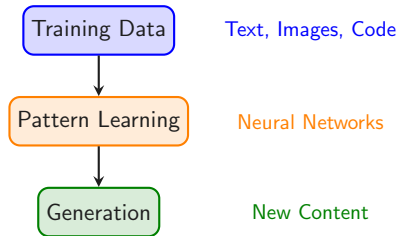
- **Creates** new content, not just analyzes
- **Learns** patterns from data
- **Generates** text, images, code, music
- **Examples:**
 - ChatGPT, Claude, Gemini
 - DALL-E, Midjourney, Stable Diffusion
 - GitHub Copilot, Codex



How Generative AI Works

The Process

1. **Training** – Learn from massive data
2. **Pattern Recognition** – Identify relationships
3. **Generation** – Create new content
4. **Refinement** – Improve through feedback



Generative AI vs Traditional AI

Traditional AI

- Analyzes existing data
- Makes predictions
- Classifies input
- **Example:** Spam detection

Generative AI

- Creates new content
- Generates novel outputs
- Produces text, images, code
- **Example:** ChatGPT writing emails

Traditional AI	Generative AI
Discriminative	Generative
Classifies	Creates
Predicts	Produces
Analyzes	Synthesizes

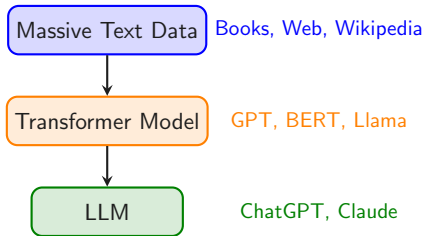
Large Language Models

What are Large Language Models (LLMs)?

Large Language Models are AI models trained on massive amounts of **text data** to understand, generate, and manipulate human language.

Key Characteristics

- Billions of Parameters
- Trained on Massive Text
- Contextual Understanding
- Few-shot & Zero-shot Learning
- Multilingual Capabilities



Major LLMs

Cloud-Based

- **OpenAI**
 - GPT-4
 - GPT-3.5
- **Anthropic**
 - Claude 3
- **Google**
 - Gemini

Open Source

- **Meta**
 - Llama 2
 - Llama 3
- **Mistral**
 - Mistral 7B
 - Mixtral
- **Hugging Face**
 - Falcon
 - Zephyr

Local & Edge

- **Ollama**
 - Llama 2/3
 - Mistral
 - Phi-2
- **LM Studio**
- **GPT4All**

Ollama

Ollama: Local LLMs

What is Ollama?

- Run LLMs locally on your machine
- Simple installation
- API for integration
- Supports popular models
- Privacy-first

Available Models

- llama2 – Meta Llama 2
- llama3 – Meta Llama 3
- mistral – Mistral 7B
- phi – Microsoft Phi-2
- gemma – Google Gemma
- llama – Multi-modal

Ollama - Installation & Setup

>_ Installation Commands

```
1 # MacOS / Linux
2 curl -fsSL https://ollama.com/install.sh | sh
3
4 # Windows
5 # Download from https://ollama.com/download/windows
6
7 # Verify installation
8 ollama --version
9
10 # Pull a model (first time)
11 ollama pull llama3
12 ollama pull mistral
13 ollama pull phi
14
15 # List downloaded models
16 ollama list
17
18 # Run a model
19 ollama run llama3
20
21 # Run with specific parameters
22 ollama run llama3 --temperature 0.7 --top-p 0.9
```

Ollama - Python Integration

Python with Ollama

```
1 import ollama
2
3 # 1. Simple generation
4 response = ollama.chat(
5     model='llama3',
6     messages=[
7         {'role': 'user', 'content': 'Explain voter turnout'}
8     ]
9 )
10 print(response['message']['content'])
11
12 # 2. Streaming response
13 stream = ollama.chat(
14     model='mistral',
15     messages=[{'role': 'user', 'content': 'What is democracy?'}],
16     stream=True
17 )
18 for chunk in stream:
19     print(chunk['message']['content'], end='', flush=True)
```

Ollama - Python Integration

</> Python with Ollama

```
1 # 3. With custom parameters
2 response = ollama.chat(
3     model='llama3',
4     messages=[{'role': 'user', 'content': 'Summarize election results'}],
5     options={
6         'temperature': 0.5,
7         'top_p': 0.9,
8         'max_tokens': 200
9     }
10 )
11
12 # 4. List available models
13 models = ollama.list()
14 print(models)
```

Ollama - Advanced Usage

Custom Models

```
1 # Create Modelfile
2 # Modelfile
3 FROM llama3
4 SYSTEM "You are an INEC election assistant."
5 PARAMETER temperature 0.3
6 PARAMETER top_p 0.9
7
8 # Build custom model
9 ollama create inec-assistant -f ./Modelfile
10
11 # Run custom model
12 ollama run inec-assistant
```

API Endpoints

- /api/generate
- /api/chat
- /api/embeddings

```
1 # Example API call
2 curl http://localhost:11434/api/chat -d '{
3   "model": "llama3",
4   "messages": [
5     {"role": "user", "content": "Hello"}
6   ]
7 }'
```

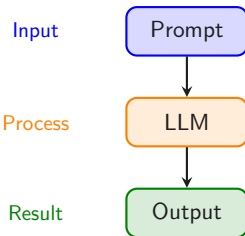
Prompt Engineering

What is Prompt Engineering?

Prompt Engineering is the art and science of designing **effective inputs** to guide AI models to produce desired outputs.

Why It Matters

- Guides LLM behavior
- Improves output quality
- Reduces hallucinations
- Enables specific tasks
- Saves time and cost



Prompt Engineering - Best Practices

Best Practices

- Be Clear and Specific
- Provide Context
- Use Examples (Few-shot)
- Define the Format
- Set the Tone
- Iterate and Refine

</> Example Comparison

```
1 # Bad Prompt
2 "Tell me about elections"
3
4 # Better Prompt
5 "Explain the Nigerian electoral process
6 in 3 paragraphs. Include: voter
7 registration, polling day, and result
8 collation."
9
10 # Best Prompt (Few-shot)
11 "Analyze the following election data:
12 LGA: Abuja, Votes: 12000, Turnout: 65%
13 LGA: Lagos, Votes: 25000, Turnout: 72%
14
15 Provide a summary and recommendations
16 in the same format for:
17 LGA: Kano, Votes: 18000, Turnout: 58%"
```

Prompt Engineering - Advanced Techniques

Role Prompting

- "Act as an INEC electoral officer..."
- Set a persona
- Define expertise
- Establish tone

Few-Shot Prompting

- Provide examples
- Show input-output pairs
- Teach by demonstration
- Improve accuracy

Chain of Thought

- "Let's think step by step..."
- Show reasoning process
- Break down problems
- Improve complex tasks

Prompt Engineering - Code Example

`</>` Complete Prompt Engineering Example

```
1 import ollama
2
3 # Role Prompting
4 role_prompt = """
5 You are an INEC electoral officer with
6 20 years of experience. Provide expert
7 analysis on election data.
8 """
9
10 # Few-shot Prompting
11 few_shot_prompt = """
12 Analyze voter turnout data:
13 Example 1: Abuja - 65% turnout - High
14 Example 2: Lagos - 45% turnout - Low
15
16 Now analyze: Kano - 55% turnout - ?
17 """
```

Prompt Engineering - Code Example

`</>` Complete Prompt Engineering Example

```
1 # Chain of Thought
2 cot_prompt = """
3 Let's think step by step:
4 1. The total registered voters in Lagos is 2,000,000
5 2. Votes cast: 1,200,000
6 3. Calculate: 1,200,000 / 2,000,000 = 0.6
7 4. Convert to percentage: 60%
8 Therefore, voter turnout in Lagos is 60%.
9 """
10
11 # Combine techniques
12 response = ollama.chat(
13     model='llama3',
14     messages=[
15         {'role': 'system', 'content': 'You are an INEC election analyst.'},
16         {'role': 'user', 'content': few_shot_prompt}
17     ]
18 )
19 print(response['message']['content'])
```

LLM Parameters

Understanding LLM Parameters

Key Parameters

- **Temperature** (0.0 - 1.0)
 - Controls randomness
 - Lower = More deterministic
 - Higher = More creative
- **Top-P** (0.0 - 1.0)
 - Nucleus sampling
 - Controls word selection
- **Max Tokens**
 - Maximum output length

Parameter Effects

- **Temperature 0.0**
 - Focused, deterministic
 - Best for factual tasks
- **Temperature 0.7**
 - Balanced, creative
 - Best for general tasks
- **Temperature 1.0**
 - Highly creative
 - Best for brainstorming

LLM Parameters - Code Example

`</>` Parameter Tuning with Ollama

```
1 import ollama
2
3 # Different temperature settings
4 prompt = "Write an election report"
5
6 # Low temperature - Factual
7 response_factual = ollama.chat(
8     model='llama3',
9     messages=[{'role': 'user', 'content': prompt}],
10    options={'temperature': 0.1}
11 )
12 print("Factual:\n", response_factual['message']['content'])
13
14 # Medium temperature - Balanced
15 response_balanced = ollama.chat(
16     model='llama3',
17     messages=[{'role': 'user', 'content': prompt}],
18     options={'temperature': 0.7}
19 )
20 print("\nBalanced:\n", response_balanced['message']['content'])
```

LLM Parameters - Code Example

`</>` Parameter Tuning with Ollama

```
1 # High temperature - Creative
2 response_creative = ollama.chat(
3     model='llama3',
4     messages=[{'role': 'user', 'content': prompt}],
5     options={'temperature': 1.0}
6 )
7 print("\nCreative:\n", response_creative['message']['content'])
8
9 # Full parameter control
10 response = ollama.chat(
11     model='llama3',
12     messages=[{'role': 'user', 'content': prompt}],
13     options={
14         'temperature': 0.5,
15         'top_p': 0.9,
16         'top_k': 50,
17         'max_tokens': 500,
18         'repeat_penalty': 1.1,
19         'stop': ['\n\n']
20     }
21 )
```

INEC Applications

Generative AI at INEC

Voter Education Chatbot

- Answer voter questions
- Multilingual support
- 24/7 availability
- Reduce call center load

Misinformation Detection

- Detect fake news
- Social media monitoring
- Fact-checking support
- Real-time alerts

Content Generation

- Generate election reports
- Summarize results
- Create voter education materials
- Translate content

Building a Voter Education Chatbot

Architecture

- **LLM** – Llama 3 / Mistral
- **Knowledge Base** – INEC documents
- **RAG** – Retrieval augmented generation
- **Interface** – Web/WhatsApp/SMS

</> Chatbot with Ollama

```
1 import ollama
2 import chromadb
3
4 # Load INEC documents
5 documents = load_inec_documents()
6 chroma_client = chromadb.Client()
7 collection = chroma_client.create_collection("
    inec_docs")
8
9 # Add documents to vector DB
10 for doc in documents:
11     collection.add(doc)
12
13 def ask_question(question):
14     # Retrieve relevant context
15     results = collection.query(question)
16     context = results['documents'][0]
```

Building a Voter Education Chatbot

Architecture

- **LLM** – Llama 3 / Mistral
- **Knowledge Base** – INEC documents
- **RAG** – Retrieval augmented generation
- **Interface** – Web/WhatsApp/SMS

</> Chatbot with Ollama

```
1  # Generate response
2  prompt = f"""
3  Context: {context}
4  Question: {question}
5  Answer based on INEC guidelines.
6  """
7
8  response = ollama.chat(
9      model='llama3',
10     messages=[{'role': 'user', 'content':
11                 prompt}]
12 )
13 return response['message']['content']
```

Key Takeaways

Key Takeaways

What We Covered

1. **Generative AI** – Creating new content
2. **LLMs** – Large language models
3. **Ollama** – Local LLM deployment
4. **Prompt Engineering** – Designing effective prompts
5. **INEC Applications** – Voter education, misinformation

💡 Key Principles

- ✓ **Start Simple** – Iterate and refine
- ✓ **Be Specific** – Clear prompts = Better results
- ✓ **Test Parameters** – Temperature, Top-P
- ✓ **Use Context** – Provide relevant information
- ✓ **Deploy Locally** – Privacy with Ollama

Resources for Further Learning

Online Resources

-  Ollama – ollama.com
-  Prompt Engineering Guide
-  Hugging Face – huggingface.co

Courses

-  DeepLearning.AI: Prompt Engineering
-  FastAI: Practical Deep Learning

Tools

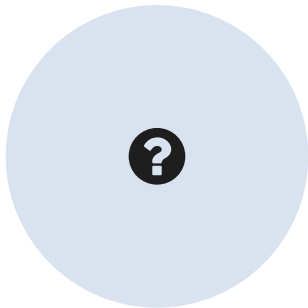
-  Ollama – Local LLMs
-  LangChain – LLM Framework
-  OpenRouter – Multi-model access

Communities

-  Ollama Discord
-  Hugging Face Forums

Thank You

Questions & Discussion



"Generative AI is the engine of modern innovation."